

Case Study Assignment: “DocTalk – Discuss Your Documents”

Scenario

You are tasked with building a **lightweight system** that allows users to **upload internal documents (PDF/DOCX/MD)** and **ask questions or discuss their content**. The system should return **grounded answers with citations**.

Core Requirements (Bare Minimum)

- Accept 1–5 documents for upload.
- Enable a simple Q&A interface (CLI or minimal UI).
- Provide answers **only from the uploaded content**.
- Include **citations** (e.g., chunk IDs or document names).
- Document your architecture and assumptions in a README.

Stretch Options (Candidate’s Choice)

- **RAG Upgrade:** Implement embeddings + vector store (FAISS or enterprise equivalent).
- **Agentic Upgrade:** Use LangGraph or similar to orchestrate tools as agents (retrieval, summarization, governance).
- **Azure-Ready Deployment:** Wrap in FastAPI, integrate AAD auth, Key Vault, and prepare for deployment on Azure App Service or Container Apps.
- **Observability:** Add basic metrics (latency, cost) and evaluation hooks.

Deliverables (End of Day)

- Source code + quickstart instructions.
- Demo script (upload → ask → show citations).
- Optional: Architecture, security, and limitations.
- Optional: Short evaluation report (groundedness, latency).
- Optional: governance checklist (e.g., Collibra entry draft).

Timebox

About 8 hours total. Allocate time for coding, evaluation, and documentation.

Evaluation Rubric

- **Baseline (40 pts):** Runs locally, citations, clean code, basic eval, governance notes.
- **Stretch (30 pts):** RAG (10), LangGraph agents (15), Azure deploy (10), Observability (5).
- **Presentation to non-technical Product Owner (30 pts):** Able to present is in an accessible way (10), able to explain details (10), presentation well formatted (10). The presentation is not part of the submission and can be share on the call.